TECNOLOGÍAS DE PROCESAMIENTO DE DATOS · ESPECIALIZACIÓN EN ANALÍTICA DE DATOS · UNIVERSIDAD RAFAEL NÚÑEZ

Taller 2A · El mes completo en BigQuery

Sesión 3 · Docente: Heyder Medrano Olier · Cada paso dice qué hacer, por qué y qué debe salir. Las capturas son de la prueba real con junio de 2025.

Antes de empezar

Palabras que vas a encontrar

Si nunca has usado estas herramientas, empieza aquí. No hace falta memorizarlas: vuelve a esta tabla cuando aparezca una palabra nueva.

Palabra	Qué significa
Nube	Usar computadores de otra empresa (Google, Amazon o Microsoft) por internet, en vez de comprar los propios. Se paga solo lo que se usa.
Servidor	Un computador potente que trabaja para otros: guarda datos o hace cálculos. No tiene pantalla; se usa por la red.
Máquina virtual	Un computador «de mentiras» creado por software dentro de un servidor real. Un servidor puede tener muchas.
Clúster	Varios computadores conectados que trabajan como si fueran uno solo para terminar más rápido.
Nodo maestro y trabajadores	En un clúster, el maestro reparte el trabajo y lleva la cuenta; los trabajadores hacen los cálculos.
IaaS, PaaS, SaaS	Infraestructura, plataforma y software «como servicio»: cuánto del sistema le entrega armado el proveedor (poco, bastante o todo).
Sin servidor (serverless)	Usted no ve ni administra máquinas: solo envía la tarea (por ejemplo, una consulta SQL) y el proveedor pone lo necesario.
BigQuery	El servicio de Google para guardar tablas enormes y consultarlas con SQL en segundos. Es sin servidor.
Conjunto de datos (dataset)	En BigQuery, una carpeta que agrupa tablas. El nuestro se llama tpd.
Tabla	Datos organizados en filas (cada viaje) y columnas (fecha, empresa, tarifa…), como una hoja de Excel gigante.

Parquet	Un formato de archivo que guarda los datos por columnas y comprimidos. Pesa menos y se lee más rápido que un CSV.
CSV	Un archivo de texto con los datos separados por comas. Es simple, pero pesado y lento para millones de filas.
SQL	El idioma para hacerle preguntas a una base de datos: SELECT (qué columnas), FROM (qué tabla), WHERE (qué filas), GROUP BY (cómo agrupar).
Consulta	Una pregunta escrita en SQL. BigQuery la ejecuta y devuelve una tabla con la respuesta.
MB, GB, TB	Medidas de tamaño: 1 GB = 1.024 MB y 1 TB = 1.024 GB. Un mes de viajes pesa unos 500 MB en Parquet.
Zona de pruebas (sandbox)	El modo gratuito de BigQuery: 10 GB de almacenamiento y 1 TB de consultas al mes, sin tarjeta.
Colab	Google Colab: cuadernos de Python que corren en la nube de Google desde el navegador, sin instalar nada.

Qué vas a lograr

Procesar **en la nube** el mes completo de viajes de Uber y Lyft de tu grupo (unos 20 millones de viajes) con **Google BigQuery**, responder con SQL las 8 preguntas del Proyecto Final y medir **cuánto cuesta** cada consulta. Es la evaluación «Simulación: procesamiento de datos masivos en la nube con AWS o Google Cloud».

Necesitas	Detalle
Una cuenta Google	Tu correo institucional o una Gmail. **No se pide tarjeta.**
Tu mes	Grupo 1: enero · Grupo 2: abril · Grupo 3: septiembre · Grupo 4: diciembre de 2025. El docente hace junio en vivo.
Unos 2,5 GB libres en tu PC	Para descargar las partes de tu mes y subirlas a BigQuery.

Elige tu mes en este tutorial

Los comandos de esta página se escriben solos con tu mes:

En el PDF aparece junio (2025_06): cambia «2025_06» por tu mes.

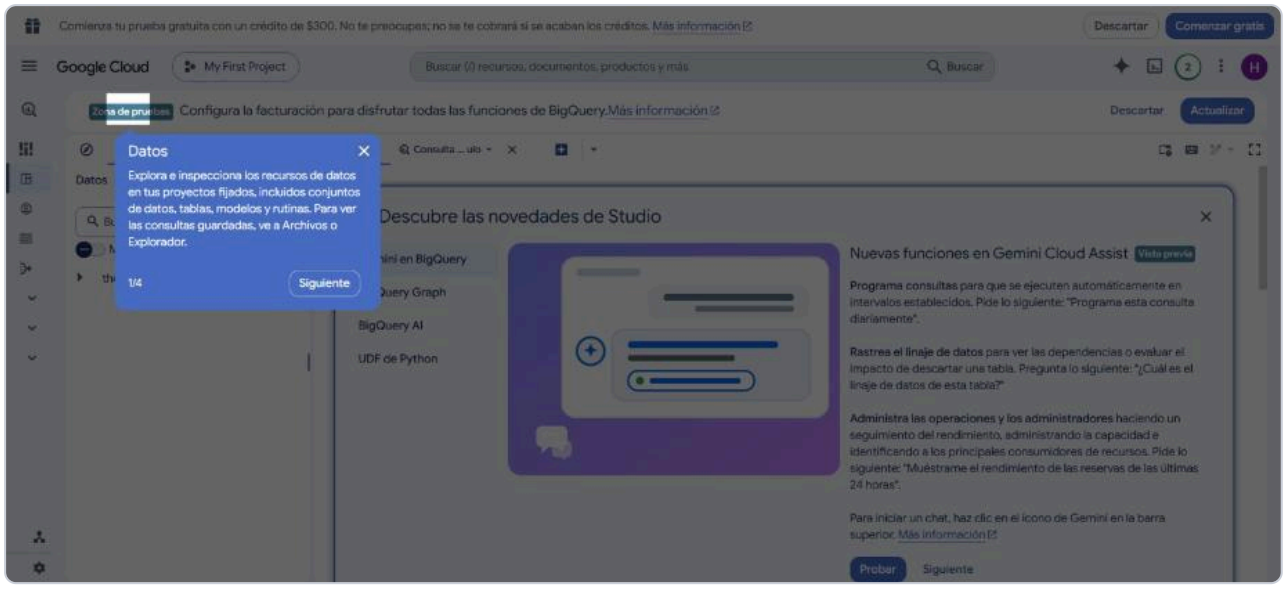
Paso 1 · BigQuery en modo gratuito (zona de pruebas)

1.1 Abrir BigQuery

> 💡 **En palabras simples:** BigQuery es una base de datos de Google en la nube. La consola es su página web de administración: desde ahí se suben datos y se escriben consultas.

Entra a **console.cloud.google.com/bigquery** con tu cuenta. Si pide aceptar los términos de Google Cloud, acéptalos. Google crea solo un proyecto llamado «My First Project».

Debe salir: arriba, la etiqueta **«Zona de pruebas»** (Sandbox) con el aviso «Configura la facturación…».



BigQuery con la etiqueta «Zona de pruebas»: gratis y sin tarjeta.

Por qué: la zona de pruebas da **10 GB de almacenamiento y 1 TB de consultas al mes** sin facturación. Las tablas caducan a los 60 días.

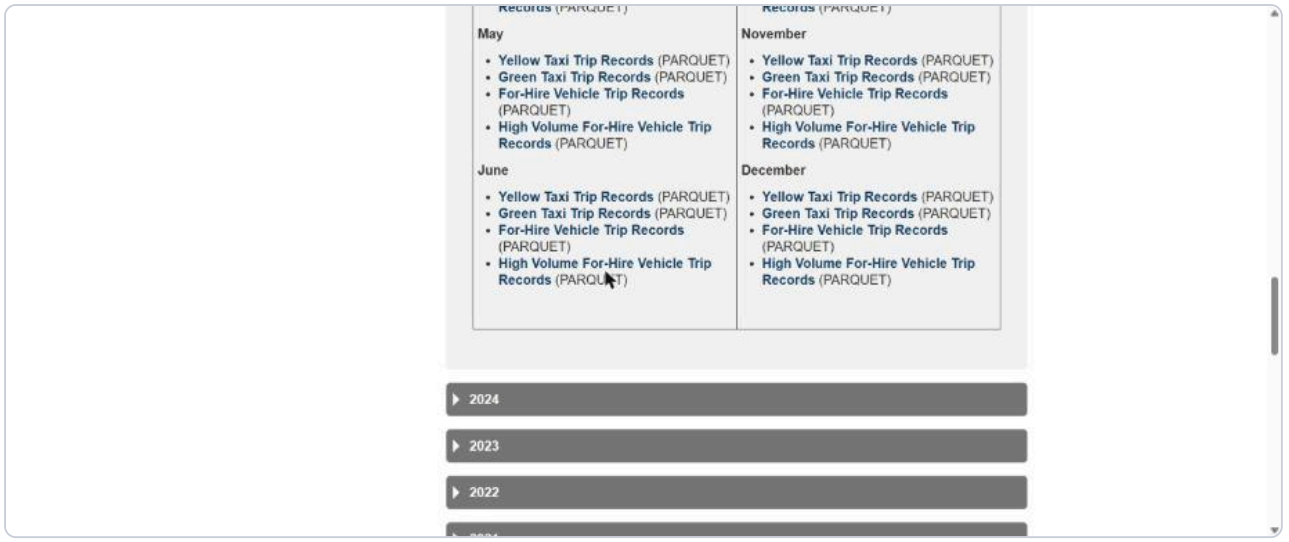
Si falla: no pulses «Comenzar gratis» ni «Actualizar»: esos botones piden tarjeta. Cierra el aviso y sigue. Si tu correo institucional dice que el servicio está desactivado, usa una Gmail.

Paso 2 · Dividir tu mes en Google Colab

2.1 De dónde salen los datos

> 💡 **En palabras simples:** La TLC es la entidad de Nueva York que regula taxis y aplicaciones como Uber y Lyft. Cada mes publica todos los viajes en un archivo gratuito.

Los publica la Comisión de Taxis y Limusinas de Nueva York (TLC) en https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page: año 2025 → tu mes → **High Volume For-Hire Vehicle Trip Records (PARQUET)**. No lo descargues a mano: el cuaderno de Colab lo baja por ti.



Página oficial de la TLC: el enlace «High Volume For-Hire Vehicle Trip Records» de cada mes.

2.2 Por qué hay que dividirlo

> 💡 **En palabras simples:** Es como llevar una mudanza en varias cajas porque no cabe en una sola: el contenido es el mismo, solo viaja en partes.

La consola de BigQuery solo deja **subir archivos de hasta 100 MB** y un mes pesa entre 460 y 535 MB. El cuaderno lo corta **por filas** en las partes que hagan falta (de unos 70 MB cada una), **sin cambiar ningún dato**, y comprueba que la suma de viajes de las partes sea el mes exacto. Es la misma idea de HDFS, que guarda los archivos en bloques de 128 MB.

2.3 Abrir el cuaderno

> 💡 **En palabras simples:** Un cuaderno de Colab es una página con bloques de código (celdas). Cada celda se ejecuta con su botón ▶ y muestra su resultado debajo.

Abre el cuaderno «dividir_mes_colab» en Colab con tu cuenta Google. Arriba a la derecha pulsa **Conectar**.

2.4 Escribir tu mes y ejecutar

> 💡 **En palabras simples:** No tienes que entender cada línea de Python: solo cambias el mes y ejecutas en orden. La tabla de abajo explica qué hace cada parte, por si quieres saberlo.

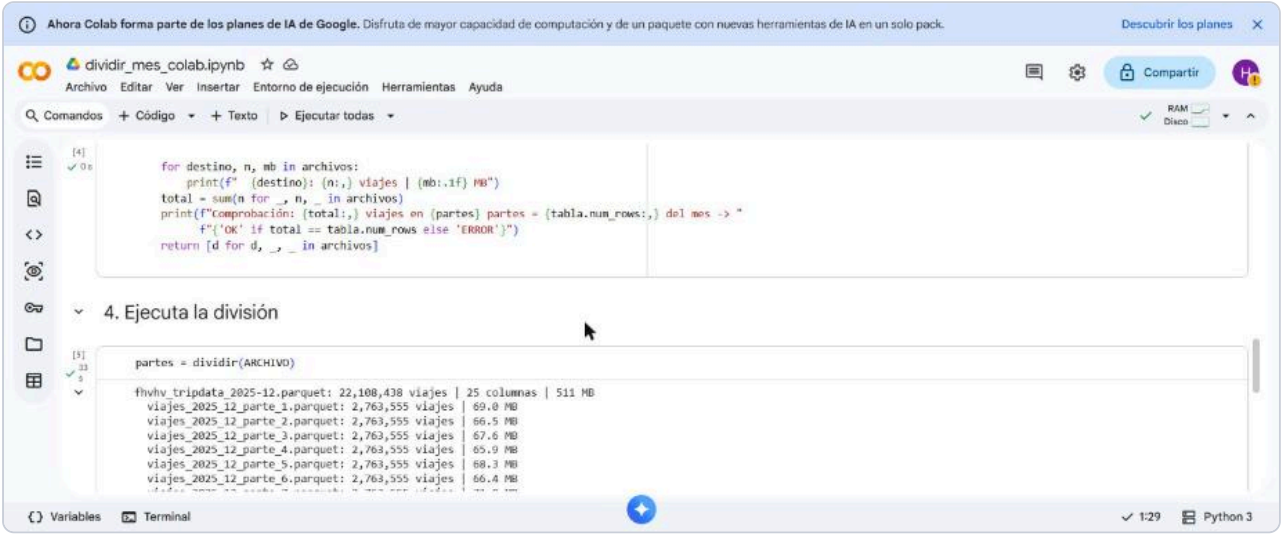
En la celda 1 cambia el mes y ejecuta cada celda en orden con su botón ▶ (o Mayús + Enter):

```
MES = "2025-06"   # <- cambia aquí el mes de tu grupo
```

La celda 2 descarga el mes (menos de un minuto), la 3 define el script y la 4 lo divide:

Línea del script	Qué hace
`pq.read_table(origen)`	lee el mes completo con pyarrow
`math.ceil(tam_mb * 1.10 / 80)`	calcula cuántas partes de unos 80 MB hacen falta (10 % de margen)
`tabla.slice(i * filas, filas)`	corta un trozo de filas, sin tocar los valores
`pq.write_table(trozo, destino, compression='snappy')`	guarda la parte en Parquet, con la misma compresión del original
`if max(mb) < 100`	si alguna parte pasa de 100 MB, vuelve a cortar con una parte más
`total == tabla.num_rows`	comprueba que no se perdió ni se repitió ningún viaje

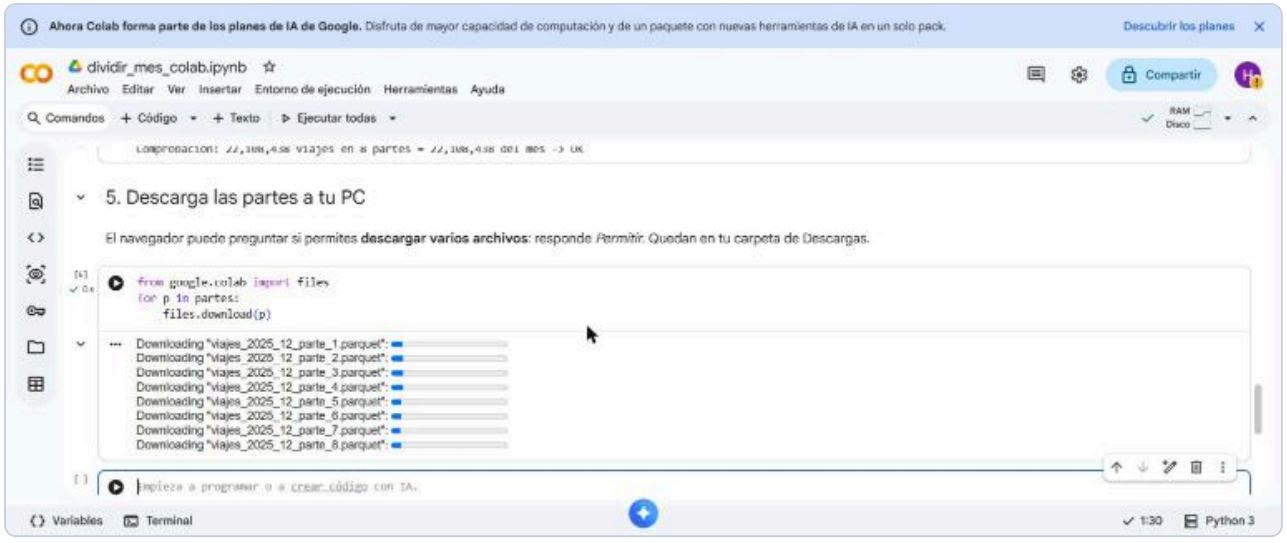
Debe salir: la lista de partes con sus MB y la línea **«Comprobación: … viajes en N partes = … del mes -> OK»**. ANOTA el número de viajes: lo vas a comparar en BigQuery.



Diciembre de 2025: 22.108.438 viajes en 8 partes de 66 a 71 MB, comprobación OK.

2.5 Descargar las partes a tu PC

Ejecuta la celda 5. El navegador pregunta si permites **descargar varios archivos**: pulsa **Permitir**. Quedan en tu carpeta Descargas como `viajes_2025_06_parte_1.parquet`, … `_parte_2`, etc.



La celda 5 descargando las 8 partes.

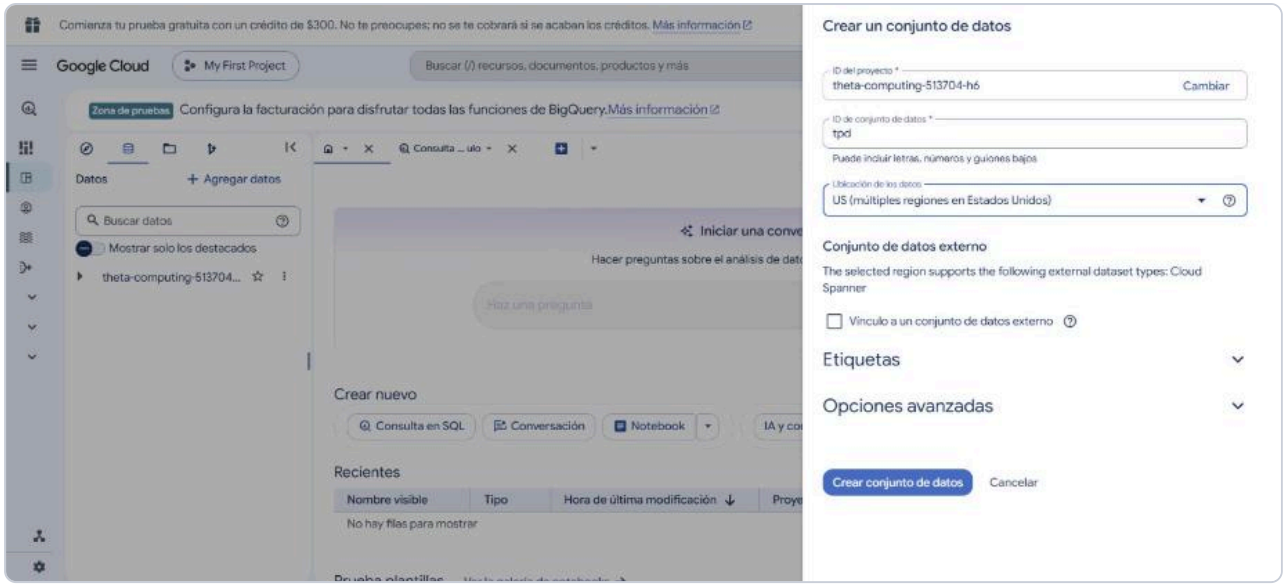
> **Si falla:** si solo bajó un archivo, el navegador bloqueó las demás descargas: permite «descargas múltiples» para colab.research.google.com (icono a la derecha de la barra de direcciones) y ejecuta otra vez la celda 5.

Paso 3 · Cargar el mes en BigQuery

3.1 Crear el conjunto de datos tpd

> 💡 **En palabras simples:** Un conjunto de datos es una carpeta dentro de BigQuery. Ahí quedan todas las tablas del taller.

En el panel izquierdo, junto al nombre de tu proyecto, pulsa los tres puntos ⋮ → **Crear un conjunto de datos**. ID: **tpd**. Ubicación: **US (múltiples regiones en Estados Unidos)**. Pulsa **Crear conjunto de datos**.



Crear el conjunto de datos tpd en la multirregión US.

Por qué: un conjunto de datos es como una base de datos: agrupa las tablas. Las tablas que se consultan juntas deben estar en la misma ubicación.

3.2 La parte 1 crea la tabla

💡 **En palabras simples:** Con la parte 1 nace la tabla: BigQuery lee el archivo y crea las columnas automáticamente (fecha, empresa, tarifa…).

Abre **tpd** → **Crear tabla** y llena así:

Campo	Valor	Por qué
Crear tabla desde	Subir	el archivo está en tu PC («Google Cloud Storage» exige facturación)
Seleccionar archivo	viajes_2025_06_parte_1.parquet	la primera parte
Formato del archivo	Parquet	es columnar y trae el esquema adentro: BigQuery no adivina tipos como con un CSV
Tabla	viajes_2025_06	un nombre por mes, por si luego cargas otros
Tipo de tabla	Tabla nativa	los datos quedan dentro de BigQuery, en su formato columnar
Partición	Sin particionar	en la zona de pruebas las particiones de más de 60 días caducan solas

Preferencia de escritura	Realiza la operación si la tabla está vacía (la que trae)	crea la tabla y no pisa datos si ya existiera

Pulsa **Crear tabla**. Tarda entre 50 y 80 segundos.



Crear tabla: Subir · Parquet · tabla del mes.

Debe salir: el aviso **«Se creó …»** y la tabla en el panel izquierdo.

3.3 Las partes 2 en adelante se AGREGAN

> 💡 **En palabras simples:** «Agrega a la tabla» significa sumar filas al final sin borrar las anteriores. Si eliges otra opción, se reemplaza lo que ya habías subido.

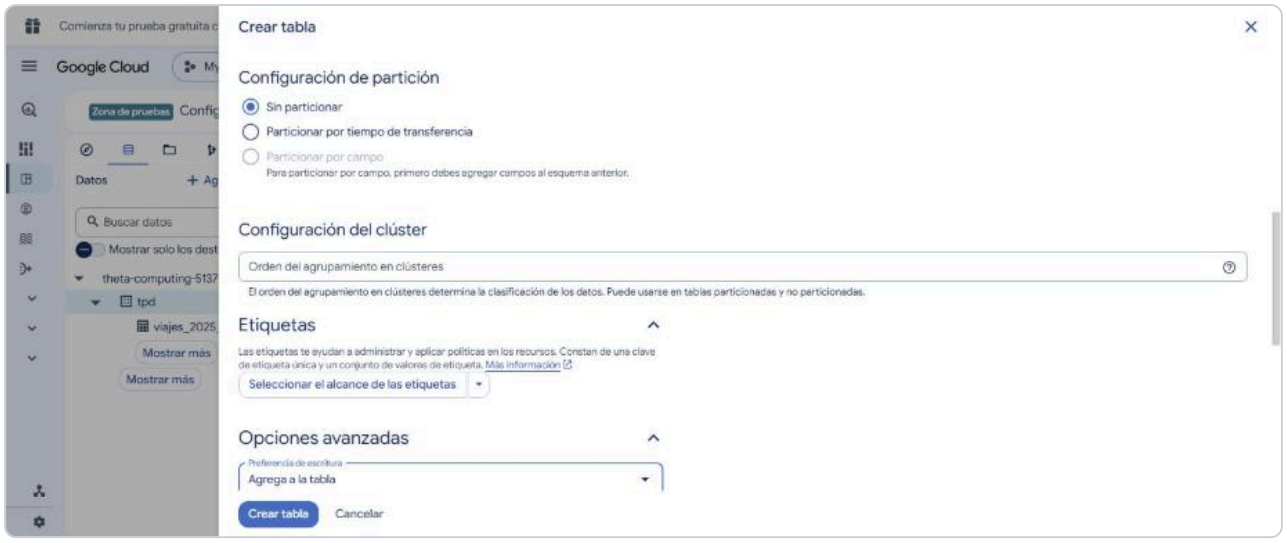
Para cada parte que falta (2, 3, 4…):

1. Junto a **tpd** pulsa los ⋮ → **Crear tabla**.
2. **Crear tabla desde:** Subir → **Seleccionar archivo** → la parte siguiente (viajes_2025_06_parte_2.parquet).
3. **Formato del archivo:** Parquet · **Conjunto de datos:** tpd.
4. **Tabla:** EXACTAMENTE el mismo nombre de la parte 1: **viajes_2025_06** (sin la palabra «parte»).
5. **Baja hasta el final** del formulario, pasando «Configuración de partición», «Configuración del clúster» y «Etiquetas».
6. Abre **«Opciones avanzadas»** (la flechita ∨ al final).
7. En **«Preferencia de escritura»** cambia «Escribir si está vacía» por **«Agregar a la tabla»**.
8. Pulsa **Crear tabla** y espera el aviso de que terminó. Luego sigue con la parte siguiente.

Debe salir: en **Detalles** de la tabla, la **Cantidad de filas** crece con cada parte que agregas.

Plan B, si «Preferencia de escritura» no aparece: sube cada parte como una tabla aparte, sin tocar opciones avanzadas (viajes_2025_06_p1, viajes_2025_06_p2, …) y luego únelas con esta consulta. El asterisco toma todas las partes:

```
CREATE OR REPLACE TABLE tpd.viajes_2025_06 AS
SELECT * FROM `tpd.viajes_2025_06_p*`
```



Opciones avanzadas → «Agrega a la tabla».

Por qué: «Agrega a la tabla» suma las filas a las que ya están. «Reemplaza la tabla» borraría lo cargado antes.

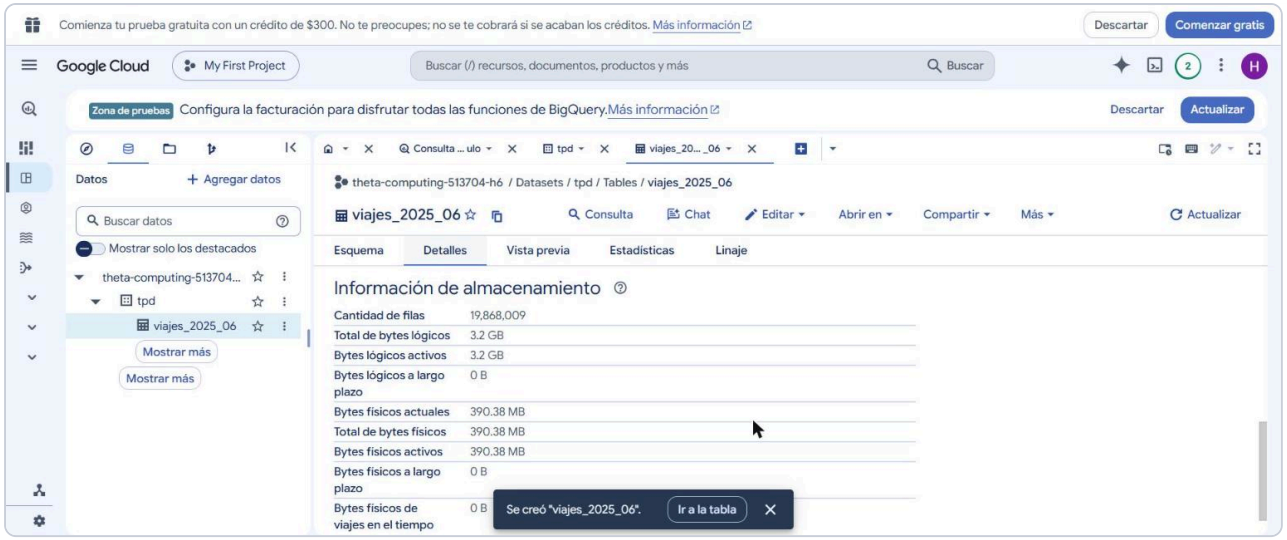
Si falla: si dice que la tabla ya existe, no elegiste «Agrega a la tabla». Si cargaste una parte dos veces, la cuenta del paso 3.4 te saldrá mayor: borra la tabla y vuelve a cargar desde la parte 1.

3.4 Comprobar que llegó el mes completo

> 💡 **En palabras simples:** Es como contar las cajas al final de la mudanza: el número de filas debe ser igual al número de viajes que dio Colab.

Abre la tabla → pestaña **Detalles** → «Información de almacenamiento».

Debe salir: Cantidad de filas igual al número de viajes que dio Colab (junio: 19.868.009). Mira también los **bytes lógicos** (junio: 3,2 GB, lo que cuenta en tus 10 GB) y los **físicos** (390 MB: BigQuery comprime por columnas).



Junio completo: 19.868.009 filas, 3,2 GB lógicos y 390 MB físicos.

3.5 La tabla de zonas

Descarga taxi_zone_lookup.csv (la tabla oficial de las 265 zonas). En **tpd** → **Crear tabla**: Subir · el CSV · formato **CSV** · tabla **zonas** · Esquema: marca **Detección automática** → Crear tabla.

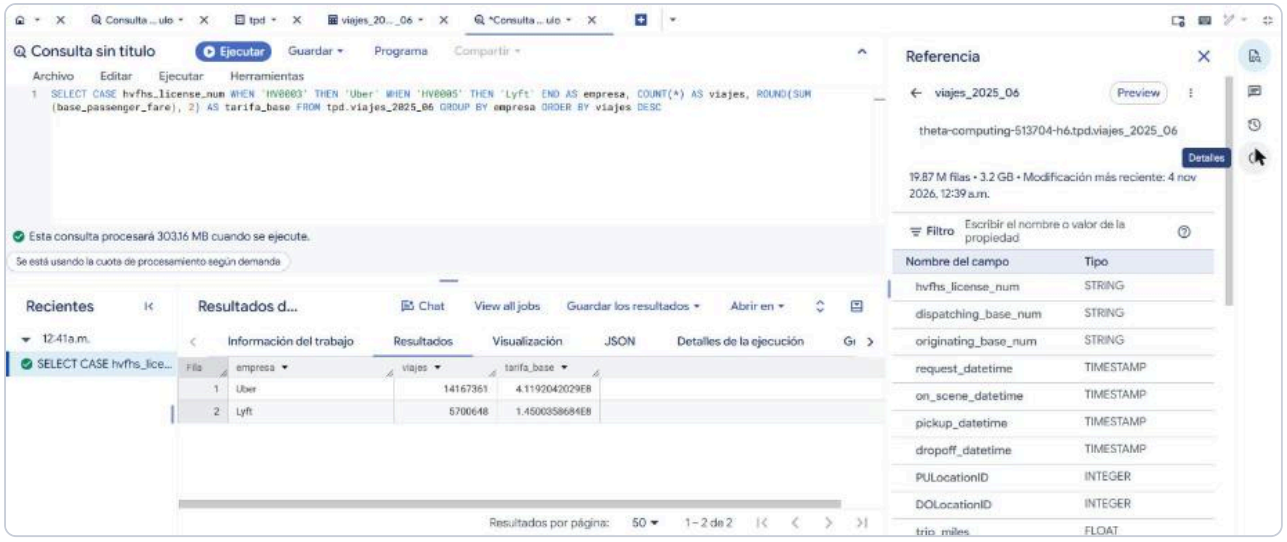
Por qué: los viajes traen el número de zona (PULocationID); esta tabla dice su nombre y su borough.

Paso 4 · Las 8 preguntas del Proyecto Final en SQL

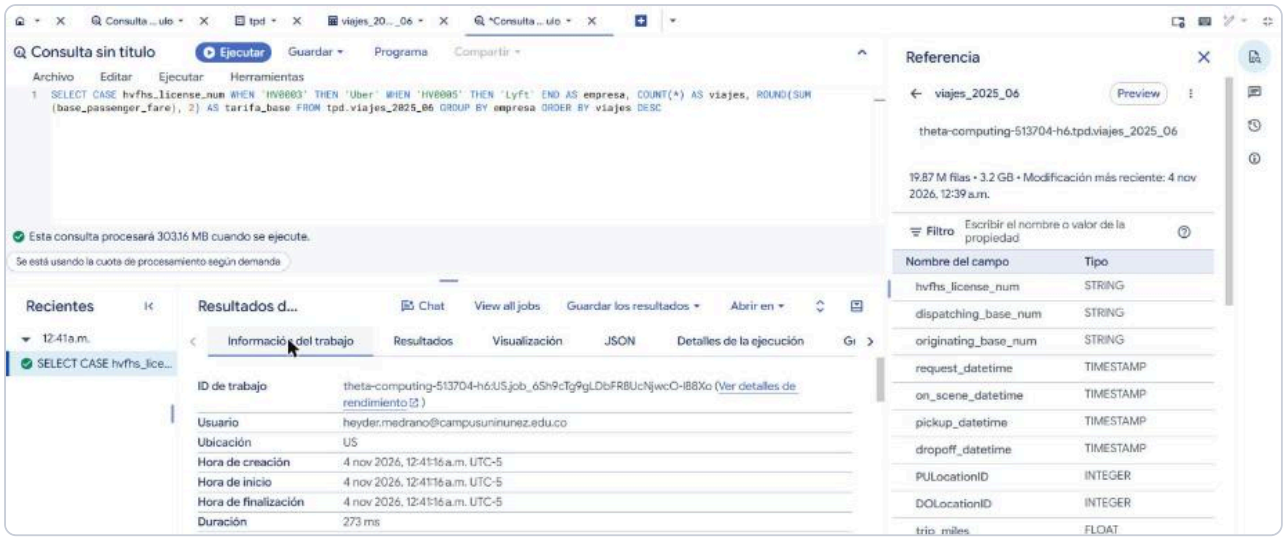
Cómo se trabaja

> 💡 **En palabras simples:** Una consulta SQL es una pregunta escrita. SELECT dice qué columnas quieres, FROM de qué tabla, WHERE qué filas y GROUP BY cómo agrupar.

Pulsa **+** (consulta nueva) o **Consulta**, pega la consulta, mira el costo estimado y pulsa **Ejecutar**. En **Información del trabajo** anota la **duración** y los **bytes procesados**. Compara las cifras con las de tu Spark o tu Hadoop y con la clave del docente: deben ser **iguales al centavo**.



Resultado: Uber 14.167.361 viajes y US$ 411.920.420,29; Lyft 5.700.648 y US$ 145.003.586,84 — lo mismo que Hadoop y Spark.



Información del trabajo: 273 ms y 303 MB procesados.

Por qué: el dinero se suma como `NUMERIC` (decimal exacto) y las millas con redondeo al par (`ROUND_HALF_EVEN`), igual que Hadoop (`round()` de Python) y Spark: así las tres tecnologías dan lo mismo al centavo.

p1 · ¿A qué horas hay más viajes y qué parte de la tarifa se queda la plataforma?

```
SELECT FORMAT('%02d', EXTRACT(HOUR FROM pickup_datetime)) AS hora, COUNT(*) AS viajes,
  SUM(CAST(base_passenger_fare AS NUMERIC)) AS tarifa_base_usd,
  SUM(CAST(base_passenger_fare AS NUMERIC)) - SUM(CAST(driver_pay AS NUMERIC)) AS
plataforma_usd,
  ROUND(100 * (SUM(CAST(base_passenger_fare AS NUMERIC)) - SUM(CAST(driver_pay AS
NUMERIC))) / SUM(CAST(base_passenger_fare AS NUMERIC)), 2) AS plataforma_pct
FROM tpd.viajes_2025_06 GROUP BY hora ORDER BY hora;
```

Antes de pulsar «Ejecutar», mira abajo del editor: «Esta consulta procesará … cuando se ejecute». ANÓTALO.

p2 · Viajes y tarifa promedio por día de la semana y empresa (DAYOFWEEK: 1 = domingo ... 7 = sábado)

```
SELECT ['7-domingo','1-lunes','2-martes','3-miercoles','4-jueves','5-viernes','6-
sabado'][OFFSET(EXTRACT(DAYOFWEEK FROM pickup_datetime) - 1)] AS dia,
  CASE hvfhs_license_num WHEN 'HV0003' THEN 'Uber' WHEN 'HV0005' THEN 'Lyft' ELSE
hvfhs_license_num END AS empresa,
  COUNT(*) AS viajes, SUM(CAST(base_passenger_fare AS NUMERIC)) AS tarifa_base_usd,
  ROUND(SUM(CAST(base_passenger_fare AS NUMERIC)) / COUNT(*), 2) AS
tarifa_promedio_usd
FROM tpd.viajes_2025_06 GROUP BY dia, empresa ORDER BY dia, empresa;
```

Antes de pulsar «Ejecutar», mira abajo del editor: «Esta consulta procesará ... cuando se ejecute». ANÓTALO.

p3 · Propinas por borough de origen y empresa

```
SELECT IFNULL(IF(z.Borough = 'N/A', 'Fuera de NYC', z.Borough), 'Desconocido') AS
borough,
  CASE v.hvfhs_license_num WHEN 'HV0003' THEN 'Uber' WHEN 'HV0005' THEN 'Lyft' ELSE
v.hvfhs_license_num END AS empresa,
  COUNT(*) AS viajes, COUNTIF(v.tips > 0) AS viajes_con_propina,
  ROUND(100 * COUNTIF(v.tips > 0) / COUNT(*), 2) AS pct_con_propina,
  SUM(CAST(v.tips AS NUMERIC)) AS propina_total_usd,
  ROUND(SUM(CAST(v.tips AS NUMERIC)) / COUNTIF(v.tips > 0), 2) AS
propina_promedio_cuando_hay_usd
FROM tpd.viajes_2025_06 v LEFT JOIN tpd.zonas z ON v.PULocationID = z.LocationID
GROUP BY borough, empresa ORDER BY borough, empresa;
```

Antes de pulsar «Ejecutar», mira abajo del editor: «Esta consulta procesará ... cuando se ejecute». ANÓTALO.

p4 · Aeropuertos, desde y hacia (un viaje entre dos aeropuertos cuenta en los dos sentidos)

```
WITH aero AS (SELECT * FROM UNNEST([STRUCT(1 AS id, 'Newark' AS nombre), (132, 'JFK'),
(138, 'LaGuardia')])),
partes AS (
  SELECT a.nombre AS aeropuerto, 'desde' AS sentido, v.* FROM tpd.viajes_2025_06 v
JOIN aero a ON v.PULocationID = a.id
  UNION ALL
  SELECT a.nombre, 'hacia', v.* FROM tpd.viajes_2025_06 v JOIN aero a ON
v.DOLocationID = a.id)
SELECT aeropuerto, sentido, COUNT(*) AS viajes,
  ROUND(SUM(CAST(base_passenger_fare AS NUMERIC)) / COUNT(*), 2) AS
tarifa_promedio_usd,
  SUM(CAST(airport_fee AS NUMERIC)) AS cargo_aeropuerto_usd,
  SUM(CAST(base_passenger_fare AS NUMERIC)) - SUM(CAST(driver_pay AS NUMERIC)) AS
```

```
plataforma_usd
FROM partes GROUP BY aeropuerto, sentido ORDER BY aeropuerto, sentido;
```

Antes de pulsar «Ejecutar», mira abajo del editor: «Esta consulta procesará … cuando se ejecute». ANÓTALO.

p5 · Pago al conductor por milla, por borough y empresa (solo viajes con distancia)

```
SELECT IFNULL(IF(z.Borough = 'N/A', 'Fuera de NYC', z.Borough), 'Desconocido') AS
borough,
  CASE v.hvfhs_license_num WHEN 'HV0003' THEN 'Uber' WHEN 'HV0005' THEN 'Lyft' ELSE
v.hvfhs_license_num END AS empresa,
  COUNT(*) AS viajes, SUM(CAST(v.driver_pay AS NUMERIC)) AS pago_conductor_usd,
  (SUM(ROUND(CAST(v.trip_miles * 100 AS BIGNUMERIC), 0, "ROUND_HALF_EVEN")) / 100) AS
millas,
  ROUND(SUM(CAST(v.driver_pay AS NUMERIC)) / (SUM(ROUND(CAST(v.trip_miles * 100 AS
BIGNUMERIC), 0, "ROUND_HALF_EVEN")) / 100), 2) AS pago_por_milla_usd
FROM tpd.viajes_2025_06 v LEFT JOIN tpd.zonas z ON v.PULocationID = z.LocationID
WHERE ROUND(CAST(v.trip_miles * 100 AS BIGNUMERIC), 0, "ROUND_HALF_EVEN") > 0
GROUP BY borough, empresa ORDER BY borough, empresa;
```

Antes de pulsar «Ejecutar», mira abajo del editor: «Esta consulta procesará … cuando se ejecute». ANÓTALO.

p6 · ¿A qué horas es más lento moverse? (velocidad = millas / horas)

```
SELECT FORMAT('%02d', EXTRACT(HOUR FROM pickup_datetime)) AS hora, COUNT(*) AS viajes,
  (SUM(ROUND(CAST(trip_miles * 100 AS BIGNUMERIC), 0, "ROUND_HALF_EVEN")) / 100) AS
millas, SUM(trip_time) / 3600 AS horas_de_viaje,
  ROUND((SUM(ROUND(CAST(trip_miles * 100 AS BIGNUMERIC), 0, "ROUND_HALF_EVEN")) / 100)
/ (SUM(trip_time) / 3600), 2) AS velocidad_mph,
  ROUND(SUM(trip_time) / COUNT(*) / 60, 2) AS duracion_promedio_min
FROM tpd.viajes_2025_06 WHERE ROUND(CAST(trip_miles * 100 AS BIGNUMERIC), 0,
"ROUND_HALF_EVEN") > 0 AND trip_time > 0
GROUP BY hora ORDER BY hora;
```

Antes de pulsar «Ejecutar», mira abajo del editor: «Esta consulta procesará … cuando se ejecute». ANÓTALO.

p7 · Zonas de recogida que más dinero le dejan a la plataforma

```
SELECT v.PULocationID AS zona_id, z.Zone AS zona, z.Borough AS borough, COUNT(*) AS
viajes,
  SUM(CAST(v.base_passenger_fare AS NUMERIC)) AS tarifa_base_usd,
  SUM(CAST(v.base_passenger_fare AS NUMERIC)) - SUM(CAST(v.driver_pay AS NUMERIC)) AS
plataforma_usd,
  ROUND((SUM(CAST(v.base_passenger_fare AS NUMERIC)) - SUM(CAST(v.driver_pay AS
NUMERIC))) / COUNT(*), 2) AS plataforma_por_viaje_usd
```

```
FROM tpd.viajes_2025_06 v LEFT JOIN tpd.zonas z ON v.PULocationID = z.LocationID
GROUP BY zona_id, zona, borough ORDER BY plataforma_usd DESC;
```

Antes de pulsar «Ejecutar», mira abajo del editor: «Esta consulta procesará … cuando se ejecute». ANÓTALO.

p8 · Cargo de congestión de Manhattan por hora

```
SELECT FORMAT('%02d', EXTRACT(HOUR FROM pickup_datetime)) AS hora, COUNT(*) AS viajes,
  COUNTIF(cbd_congestion_fee > 0) AS viajes_con_cargo,
  ROUND(100 * COUNTIF(cbd_congestion_fee > 0) / COUNT(*), 2) AS pct_con_cargo,
  SUM(CAST(cbd_congestion_fee AS NUMERIC)) AS recaudo_usd
FROM tpd.viajes_2025_06 GROUP BY hora ORDER BY hora;
```

Antes de pulsar «Ejecutar», mira abajo del editor: «Esta consulta procesará … cuando se ejecute». ANÓTALO.

Paso 5 · Optimización de costos

5.1 Lo que cobra BigQuery

> 💡 **En palabras simples:** BigQuery cobra por la cantidad de datos que lee, no por lo que te muestra. Pedir menos columnas = leer menos = pagar menos.

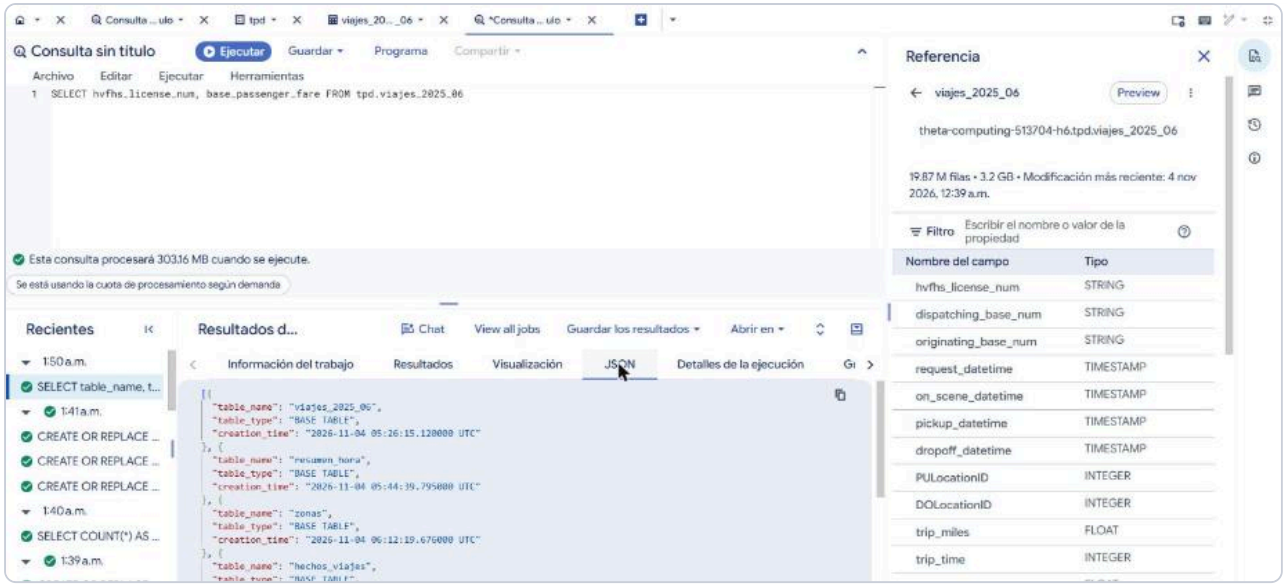
BigQuery cobra por los **bytes que lee**, no por las filas que devuelve. Pega cada consulta **sin ejecutarla** y anota lo que dice el validador:

```
SELECT * FROM tpd.viajes_2025_06
```

```
SELECT * FROM tpd.viajes_2025_06 LIMIT 10
```

```
SELECT hvfhs_license_num, base_passenger_fare FROM tpd.viajes_2025_06
```

Consulta (junio)	El validador dice	Lección
SELECT *	3,2 GB	lee las 25 columnas
SELECT * … LIMIT 10	**3,2 GB**	LIMIT no ahorra: igual lee todas las columnas
2 columnas	303,16 MB	unas 10 veces menos: elegir columnas es la optimización principal



El validador: 303,16 MB para dos columnas.

Por qué: BigQuery guarda cada columna por separado (almacenamiento columnar, como Parquet). Con 1 TB gratis al mes caben unas 3.300 consultas como la de dos columnas, pero solo unas 300 «SELECT *».

Al terminar · ¿quedó todo bien?

Verificador del taller: dice qué está bien y qué falta

Pega esta consulta en BigQuery y pulsa **Ejecutar**. No modifica nada: solo revisa tus tablas y responde fila por fila con ✓ o con lo que **falta** y en qué paso se arregla.

```
-- VERIFICADOR DEL TALLER 2A: no modifica nada, solo revisa
WITH t AS (SELECT table_id, row_count FROM tpd.__TABLES__),
esperado AS (SELECT CASE SUBSTR('2025_06', 6, 2)
  WHEN '01' THEN 20405666 WHEN '04' THEN 19753983 WHEN '06' THEN 19868009
  WHEN '09' THEN 19434641 WHEN '12' THEN 22108438 END AS viajes),
mes AS (SELECT (SELECT row_count FROM t WHERE table_id = 'viajes_2025_06') AS filas,
(SELECT viajes FROM esperado) AS viajes)
SELECT 1 AS n, 'Tabla viajes_2025_06' AS revision,
  CASE WHEN filas IS NULL THEN 'FALTA: crea la tabla con la parte 1 (paso 3.2)'
       WHEN filas = viajes THEN CONCAT('✓ mes completo: ', FORMAT('%\'d', filas), '
viajes')
       WHEN filas < viajes THEN CONCAT('FALTAN partes: tiene ', FORMAT('%\'d', filas),
' de ', FORMAT('%\'d', viajes), ' viajes (paso 3.3)')
       ELSE CONCAT('SOBRAN filas: ', FORMAT('%\'d', filas - viajes), ' de más; una
parte se subió dos veces (paso «Empezar de nuevo»)') END AS resultado
FROM mes
UNION ALL
SELECT 2, 'Tabla zonas',
  CASE WHEN (SELECT row_count FROM t WHERE table_id = 'zonas') = 265 THEN '✓ 265
zonas'
```

```
       WHEN (SELECT row_count FROM t WHERE table_id = 'zonas') IS NULL THEN 'FALTA:
carga taxi_zone_lookup.csv (paso 3.5)'
       ELSE 'REVISA: deben ser 265 filas (paso 3.5)' END
UNION ALL
SELECT 3, 'Tablas para Power BI',
  CASE WHEN (SELECT COUNT(*) FROM t WHERE table_id IN ('hechos_viajes', 'dim_zonas',
'aeropuertos')) = 3 THEN '✓ hechos_viajes, dim_zonas y aeropuertos listas'
       ELSE 'se crean en la tarde (Taller 2B, paso 1.1)' END
ORDER BY n
```

Debe salir: tres filas. Las dos primeras con ✓ al terminar la mañana; la tercera dice «se crean en la tarde» hasta que hagas el paso 1.1 del Taller 2B.

Por qué: así el grupo y el docente saben en 10 segundos si el mes quedó completo, sin contar a mano.

Si falla: si dice «Not found: Dataset … tpd», el conjunto de datos no se llama tpd o está en otro proyecto: revisa el paso 3.1. Si dice «FALTAN partes», carga las que faltan con «Agrega a la tabla». Si dice «SOBRAN filas», sigue el paso siguiente.

Empezar de nuevo (si algo salió mal)

Si subiste una parte dos veces o la tabla quedó mal, bórrala y vuelve a cargar desde la parte 1. Las partes siguen en tu carpeta Descargas: no hace falta volver a Colab.

```
DROP TABLE tpd.viajes_2025_06
```

Luego repite los pasos 3.2 (parte 1) y 3.3 (partes 2 en adelante) y vuelve a correr el verificador.

Por qué: DROP TABLE borra solo esa tabla; las demás (zonas) se quedan.

Todas las consultas juntas

Las 8 preguntas en un solo bloque, por si prefieres copiarlas de una vez. Si las ejecutas juntas, BigQuery muestra un resultado por consulta: pulsa **«Ver resultados»** en cada una.

```
-- p1 · ¿A qué horas hay más viajes y qué parte de la tarifa se queda la plataforma?
SELECT FORMAT('%02d', EXTRACT(HOUR FROM pickup_datetime)) AS hora, COUNT(*) AS viajes,
  SUM(CAST(base_passenger_fare AS NUMERIC)) AS tarifa_base_usd,
  SUM(CAST(base_passenger_fare AS NUMERIC)) - SUM(CAST(driver_pay AS NUMERIC)) AS
plataforma_usd,
  ROUND(100 * (SUM(CAST(base_passenger_fare AS NUMERIC)) - SUM(CAST(driver_pay AS
NUMERIC))) / SUM(CAST(base_passenger_fare AS NUMERIC)), 2) AS plataforma_pct
FROM tpd.viajes_2025_06 GROUP BY hora ORDER BY hora;
```

```
-- p2 · Viajes y tarifa promedio por día de la semana y empresa (DAYOFWEEK: 1 =
domingo … 7 = sábado)
SELECT ['7-domingo','1-lunes','2-martes','3-miercoles','4-jueves','5-viernes','6-
sabado'][OFFSET(EXTRACT(DAYOFWEEK FROM pickup_datetime) - 1)] AS dia,
  CASE hvfhs_license_num WHEN 'HV0003' THEN 'Uber' WHEN 'HV0005' THEN 'Lyft' ELSE
hvfhs_license_num END AS empresa,
  COUNT(*) AS viajes, SUM(CAST(base_passenger_fare AS NUMERIC)) AS tarifa_base_usd,
  ROUND(SUM(CAST(base_passenger_fare AS NUMERIC)) / COUNT(*), 2) AS
tarifa_promedio_usd
FROM tpd.viajes_2025_06 GROUP BY dia, empresa ORDER BY dia, empresa;

-- p3 · Propinas por borough de origen y empresa
SELECT IFNULL(IF(z.Borough = 'N/A', 'Fuera de NYC', z.Borough), 'Desconocido') AS
borough,
  CASE v.hvfhs_license_num WHEN 'HV0003' THEN 'Uber' WHEN 'HV0005' THEN 'Lyft' ELSE
v.hvfhs_license_num END AS empresa,
  COUNT(*) AS viajes, COUNTIF(v.tips > 0) AS viajes_con_propina,
  ROUND(100 * COUNTIF(v.tips > 0) / COUNT(*), 2) AS pct_con_propina,
  SUM(CAST(v.tips AS NUMERIC)) AS propina_total_usd,
  ROUND(SUM(CAST(v.tips AS NUMERIC)) / COUNTIF(v.tips > 0), 2) AS
propina_promedio_cuando_hay_usd
FROM tpd.viajes_2025_06 v LEFT JOIN tpd.zonas z ON v.PULocationID = z.LocationID
GROUP BY borough, empresa ORDER BY borough, empresa;

-- p4 · Aeropuertos, desde y hacia (un viaje entre dos aeropuertos cuenta en los dos
sentidos)
WITH aero AS (SELECT * FROM UNNEST([STRUCT(1 AS id, 'Newark' AS nombre), (132, 'JFK'),
(138, 'LaGuardia')])),
partes AS (
  SELECT a.nombre AS aeropuerto, 'desde' AS sentido, v.* FROM tpd.viajes_2025_06 v
JOIN aero a ON v.PULocationID = a.id
  UNION ALL
  SELECT a.nombre, 'hacia', v.* FROM tpd.viajes_2025_06 v JOIN aero a ON
v.DOLocationID = a.id)
SELECT aeropuerto, sentido, COUNT(*) AS viajes,
  ROUND(SUM(CAST(base_passenger_fare AS NUMERIC)) / COUNT(*), 2) AS
tarifa_promedio_usd,
  SUM(CAST(airport_fee AS NUMERIC)) AS cargo_aeropuerto_usd,
  SUM(CAST(base_passenger_fare AS NUMERIC)) - SUM(CAST(driver_pay AS NUMERIC)) AS
plataforma_usd
FROM partes GROUP BY aeropuerto, sentido ORDER BY aeropuerto, sentido;

-- p5 · Pago al conductor por milla, por borough y empresa (solo viajes con distancia)
SELECT IFNULL(IF(z.Borough = 'N/A', 'Fuera de NYC', z.Borough), 'Desconocido') AS
borough,
  CASE v.hvfhs_license_num WHEN 'HV0003' THEN 'Uber' WHEN 'HV0005' THEN 'Lyft' ELSE
v.hvfhs_license_num END AS empresa,
  COUNT(*) AS viajes, SUM(CAST(v.driver_pay AS NUMERIC)) AS pago_conductor_usd,
  (SUM(ROUND(CAST(v.trip_miles * 100 AS BIGNUMERIC), 0, "ROUND_HALF_EVEN")) / 100) AS
millas,
  ROUND(SUM(CAST(v.driver_pay AS NUMERIC)) / (SUM(ROUND(CAST(v.trip_miles * 100 AS
BIGNUMERIC), 0, "ROUND_HALF_EVEN")) / 100), 2) AS pago_por_milla_usd
FROM tpd.viajes_2025_06 v LEFT JOIN tpd.zonas z ON v.PULocationID = z.LocationID
```

```
WHERE ROUND(CAST(v.trip_miles * 100 AS BIGNUMERIC), 0, "ROUND_HALF_EVEN") > 0
GROUP BY borough, empresa ORDER BY borough, empresa;

-- p6 · ¿A qué horas es más lento moverse? (velocidad = millas / horas)
SELECT FORMAT('%02d', EXTRACT(HOUR FROM pickup_datetime)) AS hora, COUNT(*) AS viajes,
  (SUM(ROUND(CAST(trip_miles * 100 AS BIGNUMERIC), 0, "ROUND_HALF_EVEN")) / 100) AS
millas, SUM(trip_time) / 3600 AS horas_de_viaje,
  ROUND((SUM(ROUND(CAST(trip_miles * 100 AS BIGNUMERIC), 0, "ROUND_HALF_EVEN")) / 100)
/ (SUM(trip_time) / 3600), 2) AS velocidad_mph,
  ROUND(SUM(trip_time) / COUNT(*) / 60, 2) AS duracion_promedio_min
FROM tpd.viajes_2025_06 WHERE ROUND(CAST(trip_miles * 100 AS BIGNUMERIC), 0,
"ROUND_HALF_EVEN") > 0 AND trip_time > 0
GROUP BY hora ORDER BY hora;

-- p7 · Zonas de recogida que más dinero le dejan a la plataforma
SELECT v.PULocationID AS zona_id, z.Zone AS zona, z.Borough AS borough, COUNT(*) AS
viajes,
  SUM(CAST(v.base_passenger_fare AS NUMERIC)) AS tarifa_base_usd,
  SUM(CAST(v.base_passenger_fare AS NUMERIC)) - SUM(CAST(v.driver_pay AS NUMERIC)) AS
plataforma_usd,
  ROUND((SUM(CAST(v.base_passenger_fare AS NUMERIC)) - SUM(CAST(v.driver_pay AS
NUMERIC))) / COUNT(*), 2) AS plataforma_por_viaje_usd
FROM tpd.viajes_2025_06 v LEFT JOIN tpd.zonas z ON v.PULocationID = z.LocationID
GROUP BY zona_id, zona, borough ORDER BY plataforma_usd DESC;

-- p8 · Cargo de congestión de Manhattan por hora
SELECT FORMAT('%02d', EXTRACT(HOUR FROM pickup_datetime)) AS hora, COUNT(*) AS viajes,
  COUNTIF(cbd_congestion_fee > 0) AS viajes_con_cargo,
  ROUND(100 * COUNTIF(cbd_congestion_fee > 0) / COUNT(*), 2) AS pct_con_cargo,
  SUM(CAST(cbd_congestion_fee AS NUMERIC)) AS recaudo_usd
FROM tpd.viajes_2025_06 GROUP BY hora ORDER BY hora;
```

Al terminar · evidencia del taller

Lo que entrega el grupo (va al informe técnico de la sustentación)

Evidencia	Dónde se ve
Filas de la tabla = viajes del mes (Colab)	Detalles de la tabla
Las 8 respuestas iguales a la clave	resultado de cada consulta
Duración y bytes de cada consulta	Información del trabajo
Tabla de costos del paso 5	el validador
Comparación de tiempos: Hadoop · Spark · BigQuery	su bitácora del 31 de octubre y la de hoy